

AUTOCRASH SOS

IoT-Based Autonomous Crash Alert System

Technical Project Report

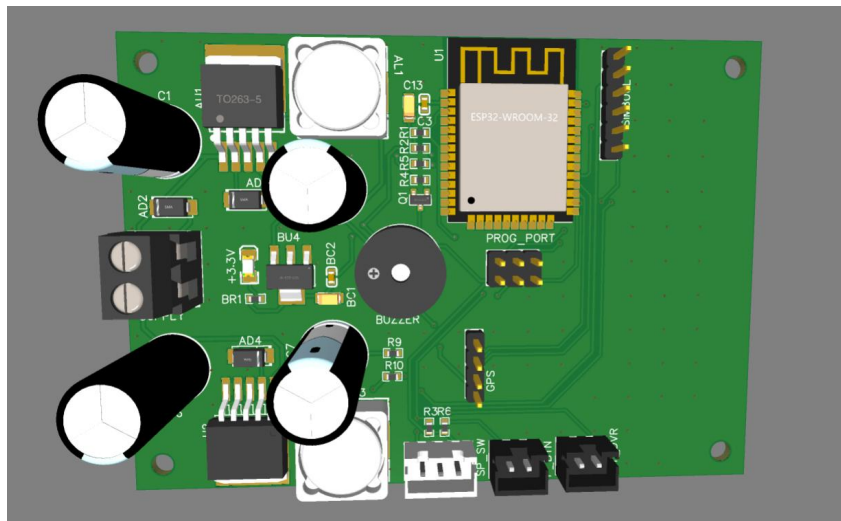


Figure 1. 3D view of the AutoCrash SOS PCB design.

Repository: github.com/noorul23/autocrash-sos-final

Table of Contents

1. Executive Summary	3
2. Project Background and Motivation.....	3
3. System Objectives	3
4. Overall System Description.....	3
4.1 Functional Blocks	4
5. Hardware Architecture	5
5.1 Circuit and Wiring Diagram	6
6. Firmware Architecture and State Machine.....	6
6.1 Firmware State Transition Logic	7
7. Electrical Design and Circuit Implementation.....	8
7.1 Pin Allocation from Firmware	9
8. PCB Design and Hardware Integration.....	10
9. Physical Prototype	12
10. Communication and Cloud Data Flow	13
10.1 GSM Notification	14
10.2 Firestore and Realtime Database Data	14
11. Ambulance Response Application	15
12. System Operation Sequence.....	16
13. Firmware Implementation Details	16
13.1 Software Libraries.....	16
13.2 Debounce and Edge Handling	17
13.3 GPS Caching.....	17
13.4 Error/Status Reporting	17
14. Testing and Observed Results.....	18
15. Technical Analysis and Design Considerations.....	19
16. Limitations and Engineering Constraints	19
17. Future Improvements	20
18. Conclusion	20
Appendix A. Firmware Pin Map	20
Appendix B. Key Firmware Parameters	21
Appendix C. Supplied Project Assets.....	21

1. Executive Summary

AutoCrash SOS is an embedded IoT accident-alert system built around an ESP32 controller. The supplied firmware and project documentation show a collision-triggered sequence in which a YL-99 impact sensor starts an air-pump actuation stage, an airbag-inspired conductive loop is monitored for breakage, a buzzer provides a 10-second driver cancellation window, and—when the alert is not cancelled—the controller sends a GSM phone call and SMS containing a Google Maps location link. The controller also records the event through Firebase Firestore and updates Firebase Realtime Database data used by the companion ambulance-response application.

The implementation combines local sensing and actuation with Wi-Fi/Firebase connectivity, a Neo-6M GPS receiver, and a SIM800L GSM modem. The supplied hardware evidence includes a perfboard prototype, a PCB layout, a 3D PCB view, a schematic, a wiring/circuit diagram, serial-monitor captures for both alert and cancellation paths, and two ambulance-application screens.

2. Project Background and Motivation

The project addresses a practical emergency-response problem: an accident can require rapid detection, location reporting and notification without relying on the driver to manually initiate the first call for assistance. The project therefore combines local collision detection with automatic communication. The system flow around collision detection, physical actuation, GPS location acquisition, GSM communication and a mobile-based emergency notification workflow.

3. System Objectives

- Automatically detect a collision event using the YL-99 impact input.
- Activate the air-pump driver to simulate an airbag deployment action.
- Use the conductive circuit-break loop as a primary trigger point for entering the alert state.
- Provide a short driver cancellation interval using a push button and buzzer.
- Acquire the latest valid GPS position and generate a Google Maps URL.
- Use the SIM800L modem for outbound call and SMS communication.
- Log accident data in Firebase Firestore and publish live event/location data to Firebase Realtime Database.
- Expose the reported incident through the companion ambulance response application.

4. Overall System Description

The supplied architecture diagram shows the ESP32 at the center of the system. The YL-99 impact trigger, airbag actuator path and circuit-break loop connect to the controller. GPS and the push button are controller inputs. The controller communicates outward through Wi-Fi to Firebase and an application, while the SIM800L provides the GSM call/SMS path. The architecture diagram also shows the reported location being represented through Google Maps.

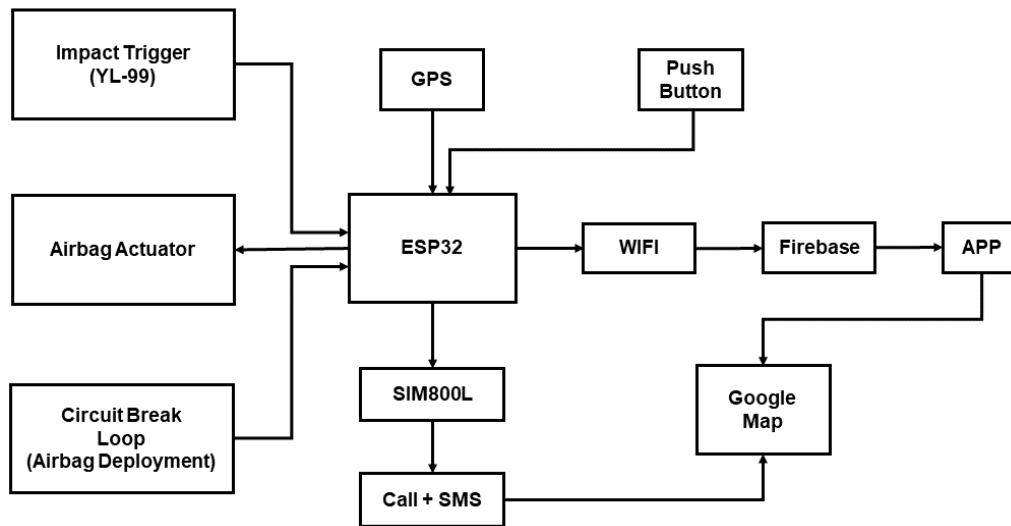


Figure 2. Supplied system architecture diagram.

4.1 Functional Blocks

Collision sensing: YL-99 produces the collision trigger input used by the ESP32 state machine.

Physical actuation: GPIO14 drives the air-pump control path. The supplied circuit diagram shows an L298N motor driver between the ESP32 control and the pump.

Event confirmation: GPIO33 monitors the detachable conductive loop with INPUT_PULLUP semantics: intact = LOW, broken = HIGH.

Driver interface: GPIO23 is a push-button input; GPIO22 drives the buzzer used during the cancellation period.

Positioning: Neo-6M GPS data is read through UART2 on GPIO16/GPIO17 and parsed by TinyGPSPlus.

GSM notification: SIM800L is connected through UART1 on GPIO4/GPIO5 and is controlled with AT commands.

Cloud logging: Wi-Fi connects the ESP32 to Firebase Firestore and Firebase Realtime Database for event logging and live data.

Ambulance application: The screenshots show a nearby-accidents interface with status, reported coordinates, time, allocation and map-related actions.

5. Hardware Architecture

The documented hardware set consists of an ESP32, SIM800L GSM module, Neo-6M GPS module, YL-99 impact sensor, detachable circuit loop, L298N driver, air pump, buzzer, push button, power-conversion circuitry and a 18650 battery in the physical prototype. The repository README lists the same principal components.

Subsystem	Component / implementation	Observed role
Controller	ESP32-WROOM-32	Central state machine, I/O, Wi-Fi/Firebase and serial interfaces
Impact sensing	YL-99	Starts the collision sequence on a rising HIGH event
Confirmation loop	Conductive loop to GND	Reports broken condition as HIGH through INPUT_PULLUP
Actuation	L298N + air pump	Drives the physical airbag-deployment simulation
Location	Neo-6M GPS	Provides latitude/longitude data via UART
GSM	SIM800L	Places call and sends SMS using AT commands
Driver interface	Push button + buzzer	Cancellation input and countdown indication
Power conversion	12 V supply / LM2596 / 3.3 V regulator stages	Provides regulated rails shown in circuit and schematic
Energy storage	18650 cell in prototype	Local battery source visible in prototype assembly

Table 1. Hardware subsystem summary.

5.1 Circuit and Wiring Diagram

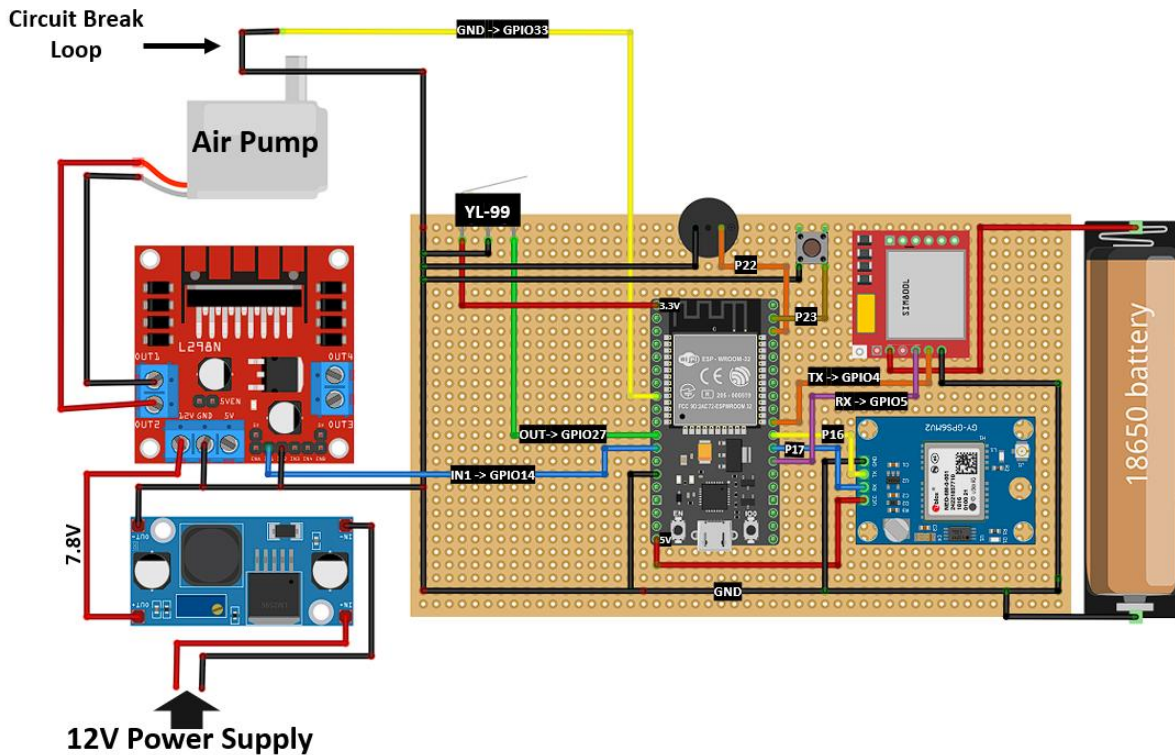


Figure 3. Supplied prototype wiring/circuit diagram showing the ESP32, YL-99, air pump, L298N, GPS, SIM800L and battery connections.

The wiring diagram shows the air pump powered through the motor-driver stage, the ESP32 mounted on a prototyping board, the YL-99 impact sensor, GPS and SIM800L, a buzzer, push button, a 7.8 V intermediate rail shown near the actuator branch, and an 18650 battery. The drawing is used here as a descriptive representation of the supplied prototype wiring rather than as a production wiring standard.

6. Firmware Architecture and State Machine

The firmware is organized as a three-state finite state machine: IDLE, PUMPING and ALERT_PENDING. The controller continuously reads GPS data and the three digital input conditions, applies a 50 ms debounce function to the collision, button and loop signals, and then executes state-dependent behavior. This is directly visible in the supplied autocrash.ino firmware.

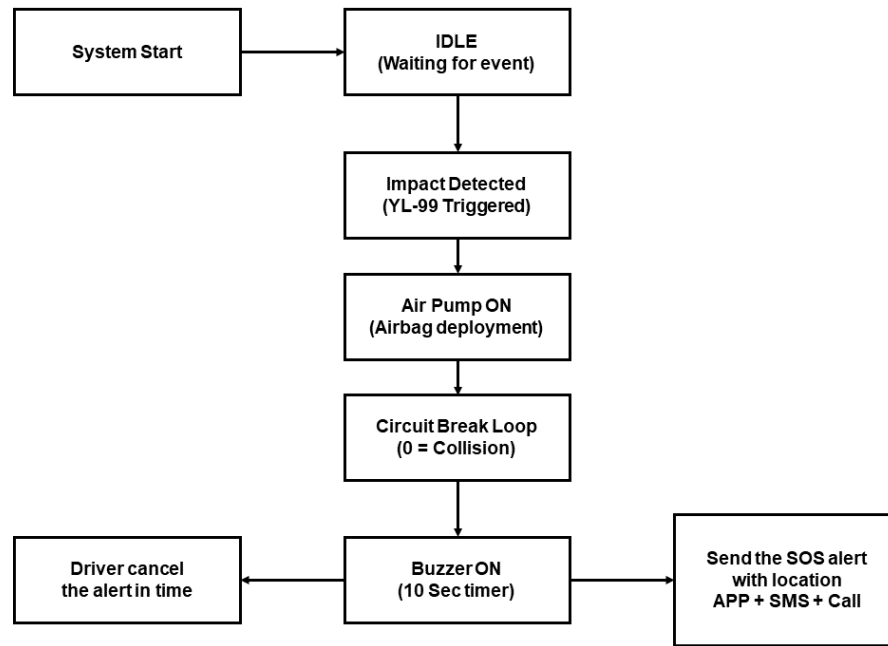


Figure 4. Supplied system flowchart for the collision-to-alert sequence.

IDLE: Waits for a new impact edge. A new impact starts the pump. If a loop-break edge is detected while idle, the system enters ALERT_PENDING immediately.

PUMPING: Keeps the air-pump control active until the loop breaks or the 6-second pump-duration timer expires. A loop break during this state immediately transitions to ALERT_PENDING and stops the pump.

ALERT_PENDING: Activates the buzzer and allows cancellation by button press for 10 seconds. When the timeout expires, the firmware sends call/SMS data and updates Firebase, then returns to IDLE.

6.1 Firmware State Transition Logic

Current state	Event	Next state	Action
IDLE	Collision stable edge	PUMPING	MOTOR_PIN = HIGH; pumpStartTime recorded
PUMPING	Loop breaks	ALERT_PENDING	Motor LOW; buzzer HIGH; alert timer starts

Current state	Event	Next state	Action
PUMPING	6 s elapsed	IDLE	Motor LOW; waits for next event
ALERT_PENDING	Button press during 10 s	IDLE	Buzzer LOW; alert cancelled
ALERT_PENDING	10 s elapsed	IDLE	Call/SMS + Firestore + RTDB update

Table 2. Firmware state transition logic.

7. Electrical Design and Circuit Implementation

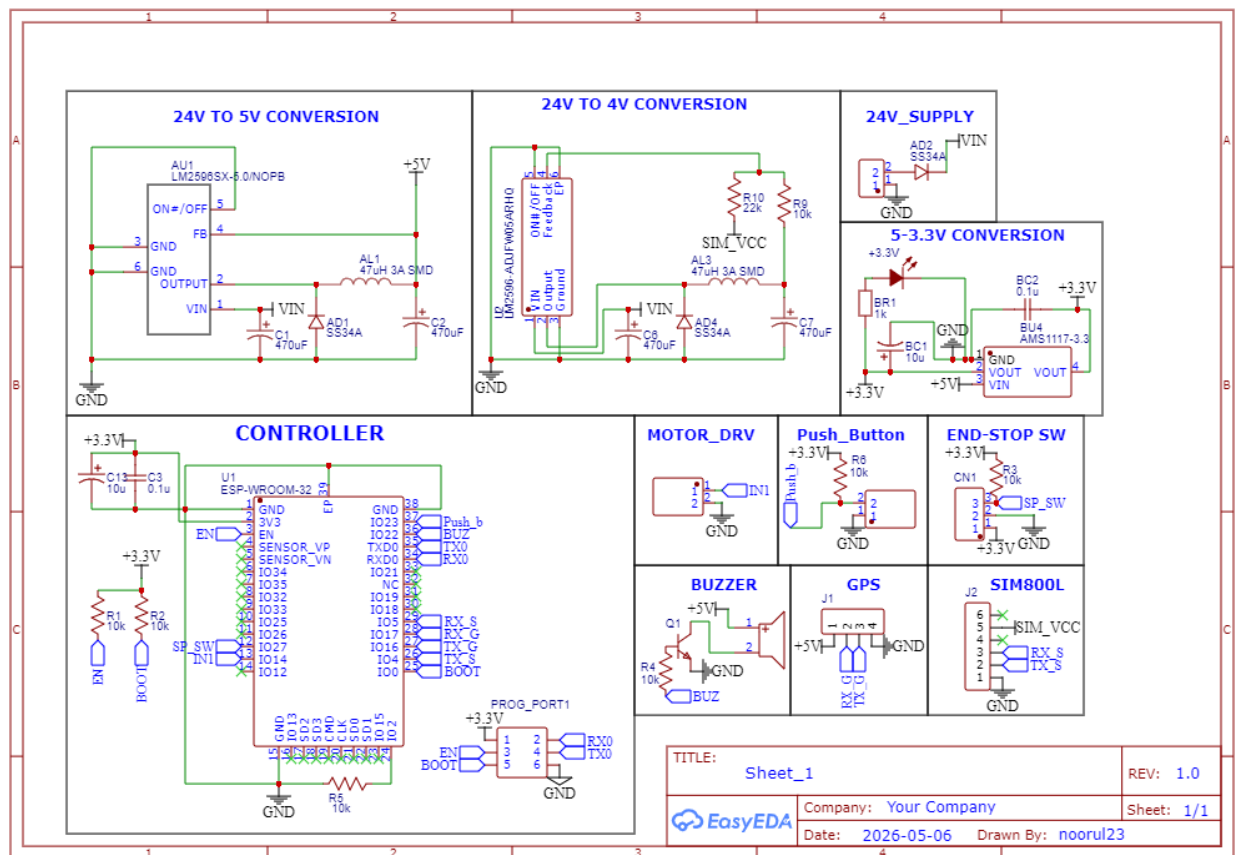


Figure 5. Supplied schematic showing power conversion, controller, motor driver, push button, end-stop, buzzer, GPS and SIM800L interface blocks.

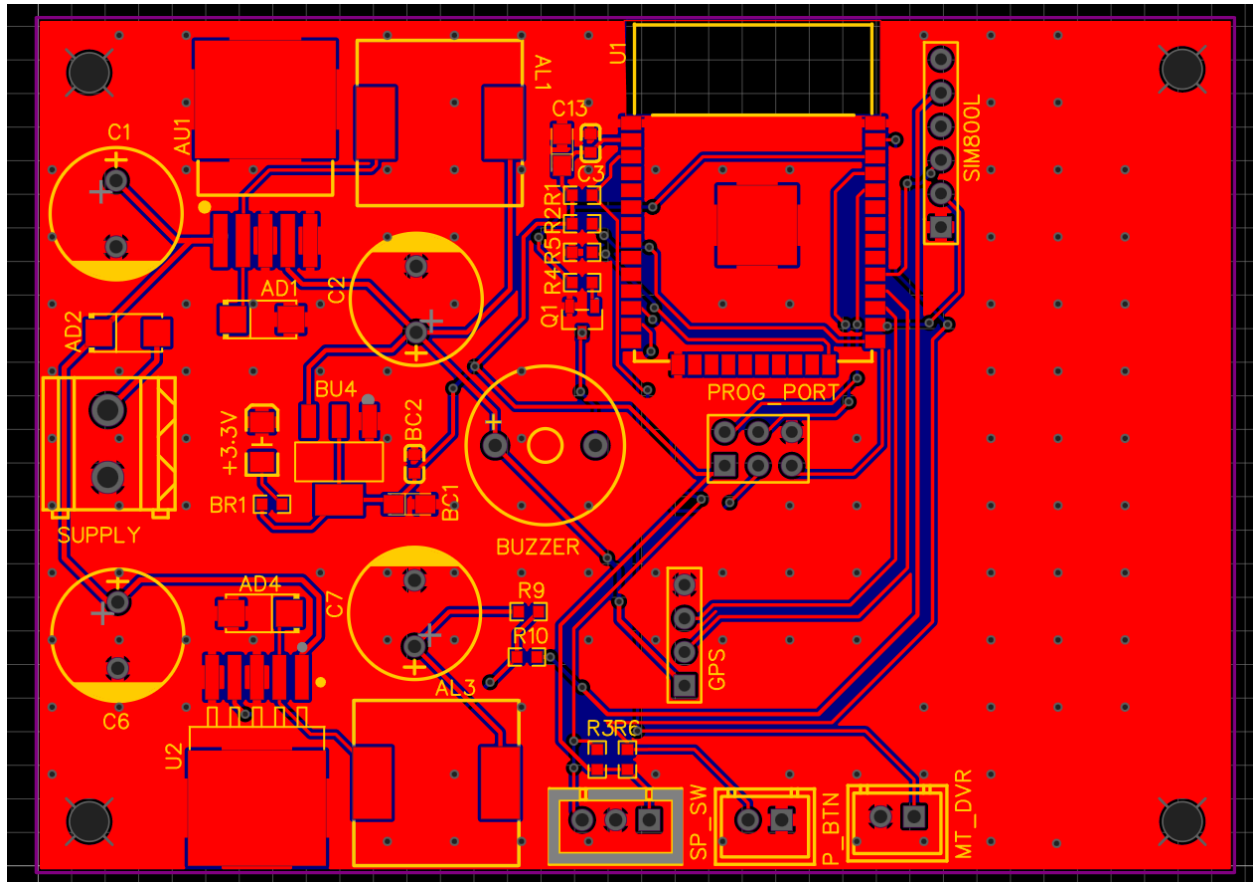
The schematic is divided into power conversion and functional interface blocks. It shows a 24 V-to-5 V conversion stage, a 24 V-to-4 V conversion stage, a 24 V supply path, a 3.3 V conversion stage, the ESP32 controller, motor-driver interface, push-button input, end-stop/circuit-break interface, buzzer stage, GPS interface and SIM800L interface. The controller portion exposes the same logical signals used by the firmware, including buzzer, motor-driver, switch/loop and serial connections.

7.1 Pin Allocation from Firmware

Pin	Firmware symbol	Function
GPIO 16	GPS_RX	UART2 receive from GPS
GPIO 17	GPS_TX	UART2 transmit to GPS
GPIO 4	SIM800L_RX	UART1 receive from GSM module
GPIO 5	SIM800L_TX	UART1 transmit to GSM module
GPIO 22	BUZZER_PIN	Buzzer output
GPIO 23	BUTTON_PIN	Push button, INPUT_PULLUP; pressed = LOW
GPIO 14	MOTOR_PIN	Air-pump driver output
GPIO 27	SWITCH_PIN	YL-99 impact input; active HIGH
GPIO 33	LOOP_PIN	Circuit-break loop; intact LOW, broken HIGH

Table 3. Pin allocation from firmware.

8. PCB Design and Hardware Integration



The PCB layout consolidates the controller and interface circuitry into a single board design. Visible silkscreen labels include the ESP32 module footprint, power section, buzzer area, programming header, GPS connector area and other I/O/connectors. Routing is shown on the board together with through-hole mounting points and component footprints.

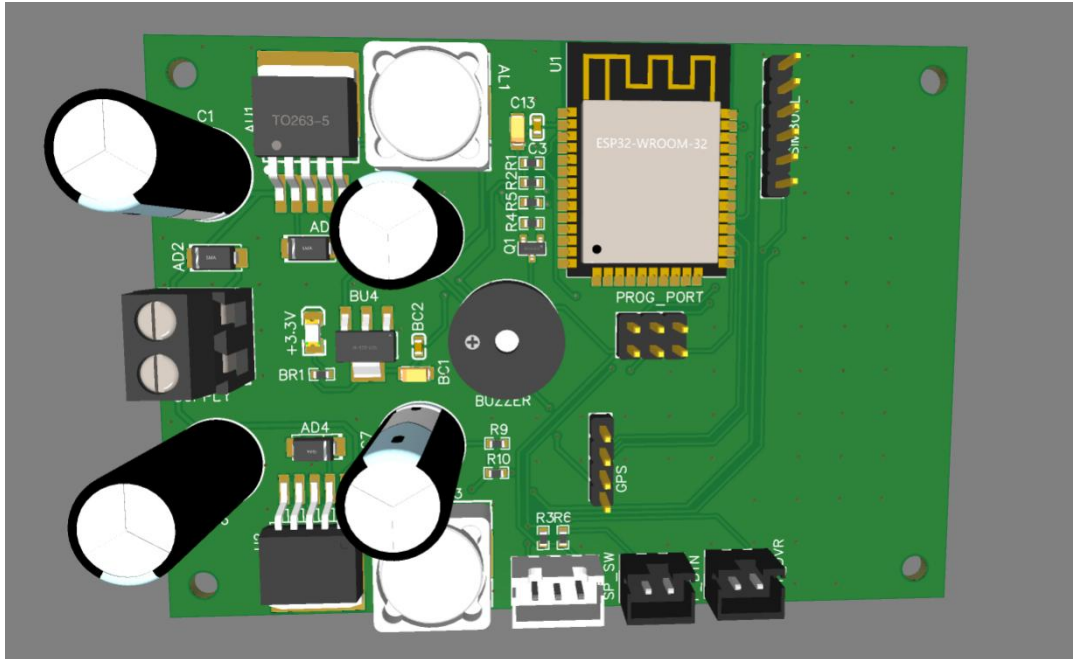


Figure 7. 3D rendering of the PCB and major component placements.

The 3D view makes the intended component placement more apparent: the ESP32 module is positioned near the upper-right controller section, the buzzer is near the center, the power and conversion components occupy the left side, and external connectors are located around the board edges. These placements correspond to the PCB layout labels and the supplied schematic.

9. Physical Prototype

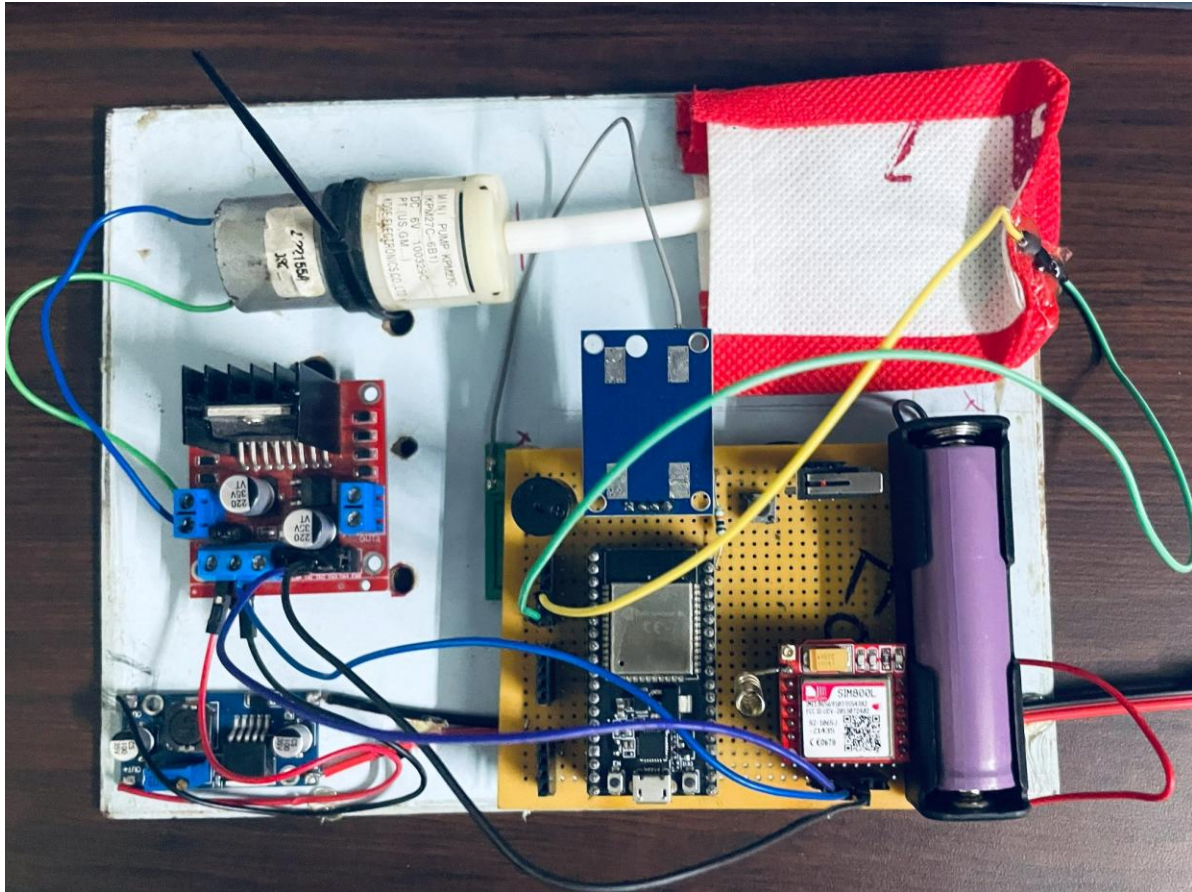


Figure 8. Physical prototype showing the air-pump/airbag mechanism, motor driver, power electronics, controller, GPS/GSM modules and battery.

The prototype photograph demonstrates an assembled system using a perforated-board controller section with discrete wiring, the motor driver and DC air pump, a fabric airbag-like enclosure, an 18650 battery and the GPS/GSM modules.

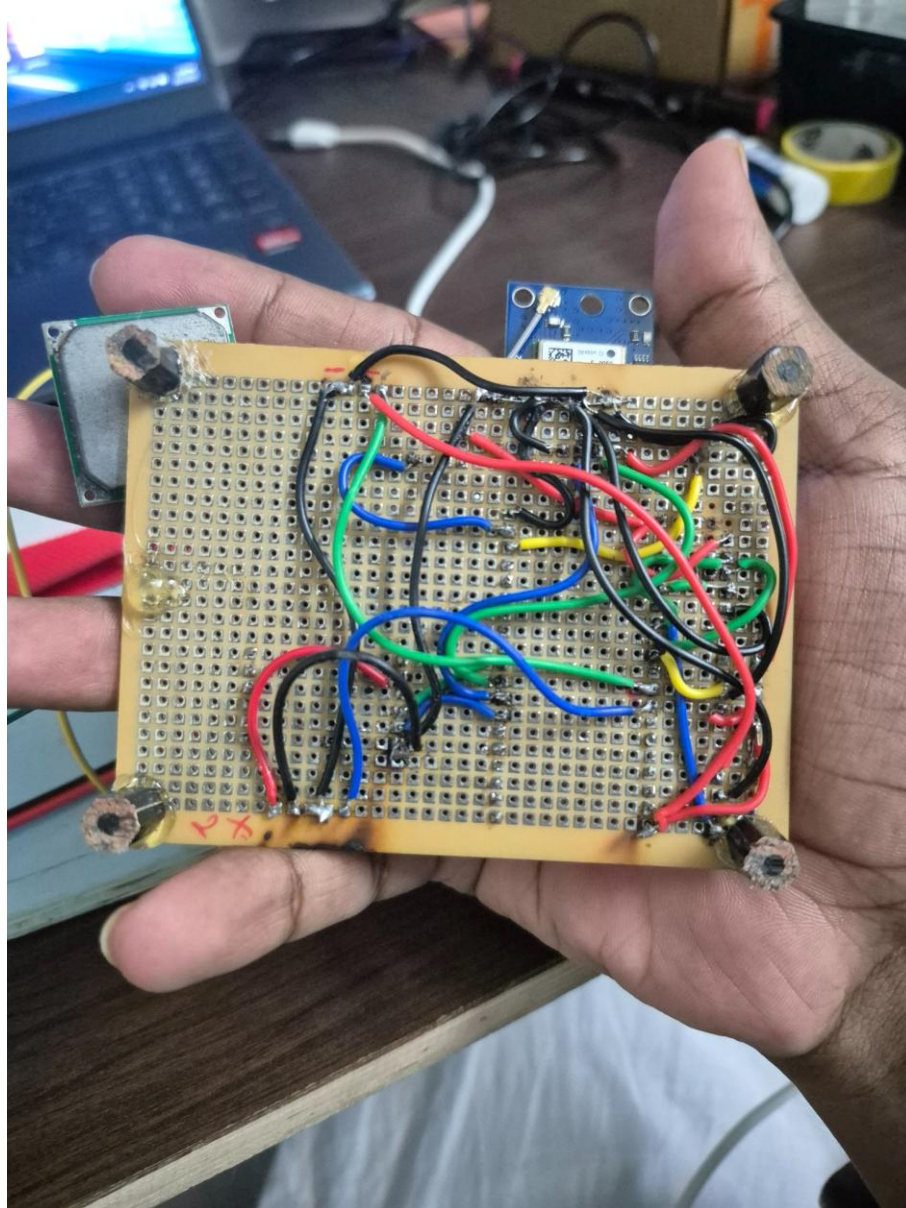


Figure 9. Rear side of the prototyping board showing point-to-point/perfboard wiring.

The rear-side photograph shows extensive point-to-point wiring across the perforated board. This supports the interpretation that the demonstrated hardware is a prototype assembly rather than a finalized production wiring harness.

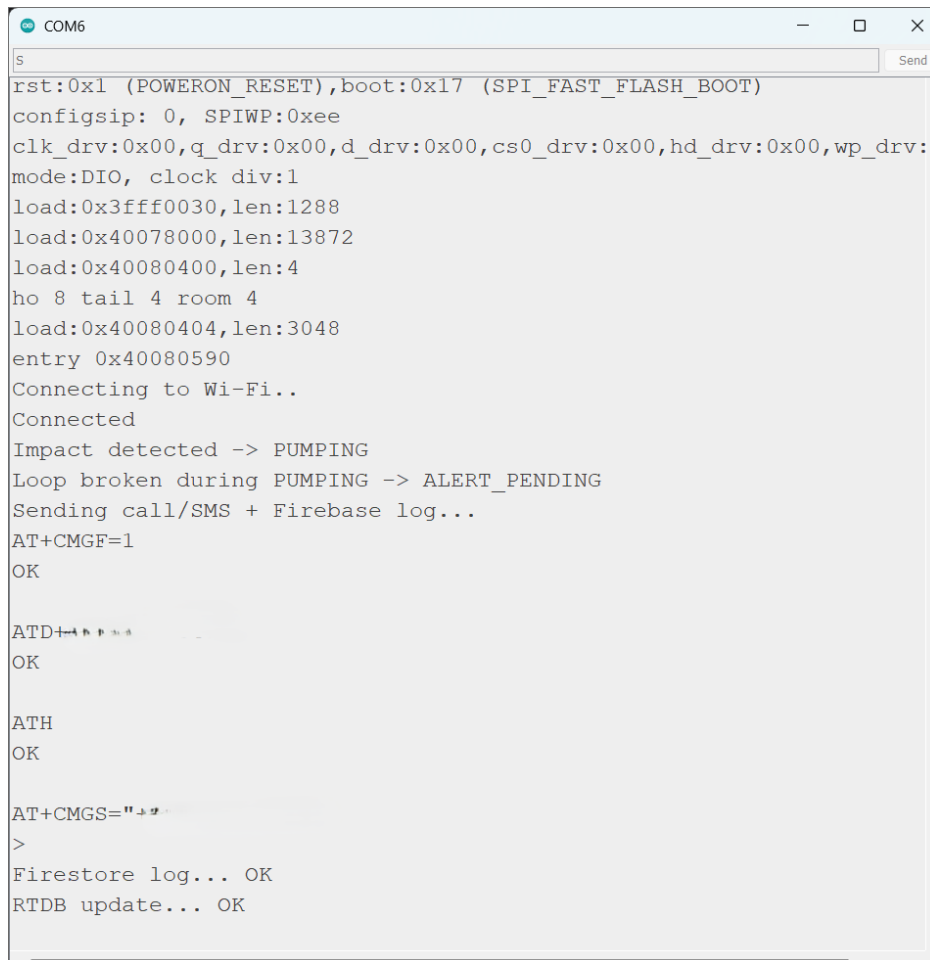
10. Communication and Cloud Data Flow

The firmware establishes Wi-Fi during setup and uses network time through `configTime(19800, 0, "pool.ntp.org")` for timestamps. Firebase configuration is initialized through `Firebase_ESP_Client`. At alert timeout, the firmware calls the SMS/call routine and then creates a

Firestore document plus a Realtime Database JSON update. The repository README also identifies Firebase-backed application communication as part of the system architecture.

10.1 GSM Notification

The function `sendSMSWithLocation()` creates a Google Maps query URL from the current latitude and longitude. It then selects text mode with `AT+CMGF=1`, performs a phone call using `ATD...`, hangs up with `ATH`, and sends the alert message through `AT+CMGS` followed by `CTRL+Z`. The supplied serial-monitor alert image captures this operational path, including the GSM command responses and the subsequent Firebase log/update messages.



```
COM6
S
rst:0x1 (POWERON_RESET),boot:0x17 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:
mode:DIO, clock div:1
load:0x3fff0030,len:1288
load:0x40078000,len:13872
load:0x40080400,len:4
ho 8 tail 4 room 4
load:0x40080404,len:3048
entry 0x40080590
Connecting to Wi-Fi..
Connected
Impact detected -> PUMPING
Loop broken during PUMPING -> ALERT_PENDING
Sending call/SMS + Firebase log...
AT+CMGF=1
OK

ATD+1415551212
OK

ATH
OK

AT+CMGS="1415551212"
>
Firestore log... OK
RTDB update... OK
```

Figure 10. Serial-monitor evidence of a triggered alert, GSM activity, and Firebase updates.

10.2 Firestore and Realtime Database Data

The Firestore logging function creates an accident document under an `accidents/<random-id>` path. The recorded fields include `accident-status`, `fire-status`, `ambulance-status`, `latitude`, `longitude` and `timestamp`. The Realtime Database function updates a `/liveData` JSON object containing accident status, fire status, location latitude/longitude and a millisecond-based timestamp. These

fields align with the accident-status, location and ambulance-status information visible in the application screenshots.

11. Ambulance Response Application

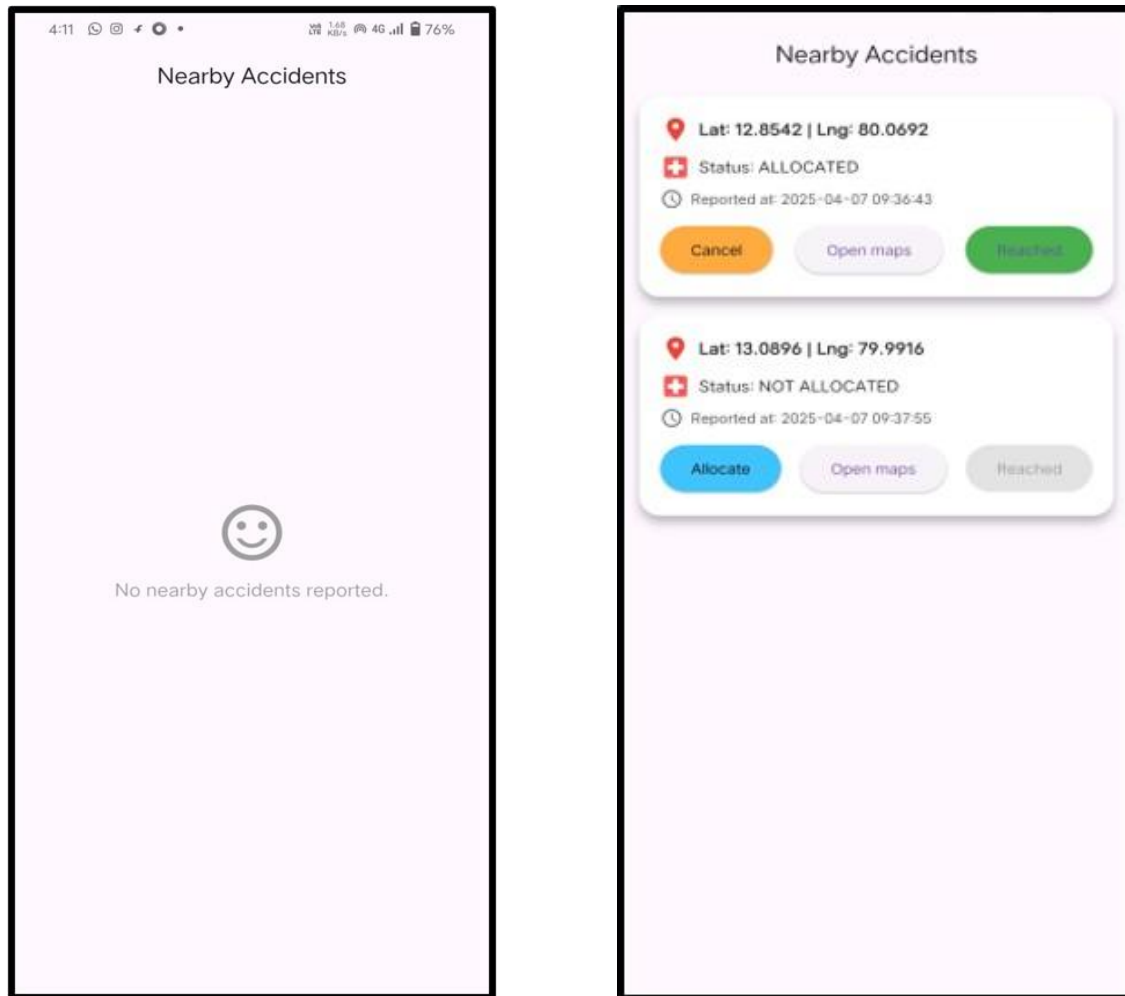


Figure 11. Ambulance application screens.

The two supplied application screenshots show a "Nearby Accidents" interface. One state presents no reported incidents. The second state lists reported incidents with latitude/longitude, status, report time, and action controls including allocation, map opening and reached/cancel-related actions. The application is intended to receive and display GPS-based emergency alerts and provides accident allocation and location/navigation functionality.

12. System Operation Sequence

Step	Stage	Technical behavior
1	Power-up and connectivity	ESP32 starts serial interfaces, connects to Wi-Fi, initializes NTP/Firebase, configures GPS and SIM800L UARTs, and initializes GPIO states.
2	Idle monitoring	The main loop continuously reads GPS data and debounced collision, button and loop signals.
3	Impact detection	A new stable HIGH transition at the YL-99 input starts the air-pump drive and enters PUMPING.
4	Airbag actuation	The motor/pump output remains active for up to 6 seconds or until the circuit-break loop opens.
5	Crash confirmation	When the loop becomes broken, the controller stops the pump, activates the buzzer and enters ALERT_PENDING.
6	Driver cancellation	A push-button press during the 10-second window returns the system to IDLE without sending the emergency alert.
7	Emergency notification	If the 10-second period expires, the controller calls/SMSes the configured contact with a Google Maps location link.
8	Cloud logging	The controller creates a Firestore accident record and updates the Firebase Realtime Database /liveData node.
9	Ambulance-side response	The companion application can present the reported incident, its location and allocation/response status.

Table 4. System operation sequence.

13. Firmware Implementation Details

13.1 Software Libraries

WiFi.h: ESP32 Wi-Fi connection

Firebase_ESP_Client.h: Firebase Authentication, Firestore and RTDB operations

TinyGPS++.h: GPS sentence parsing and position extraction

HardwareSerial.h: Dedicated hardware UARTs for GPS and SIM800L

ArduinoJson.h: JSON support used with the firmware/Firebase context

time.h: Local time formatting for accident records

13.2 Debounce and Edge Handling

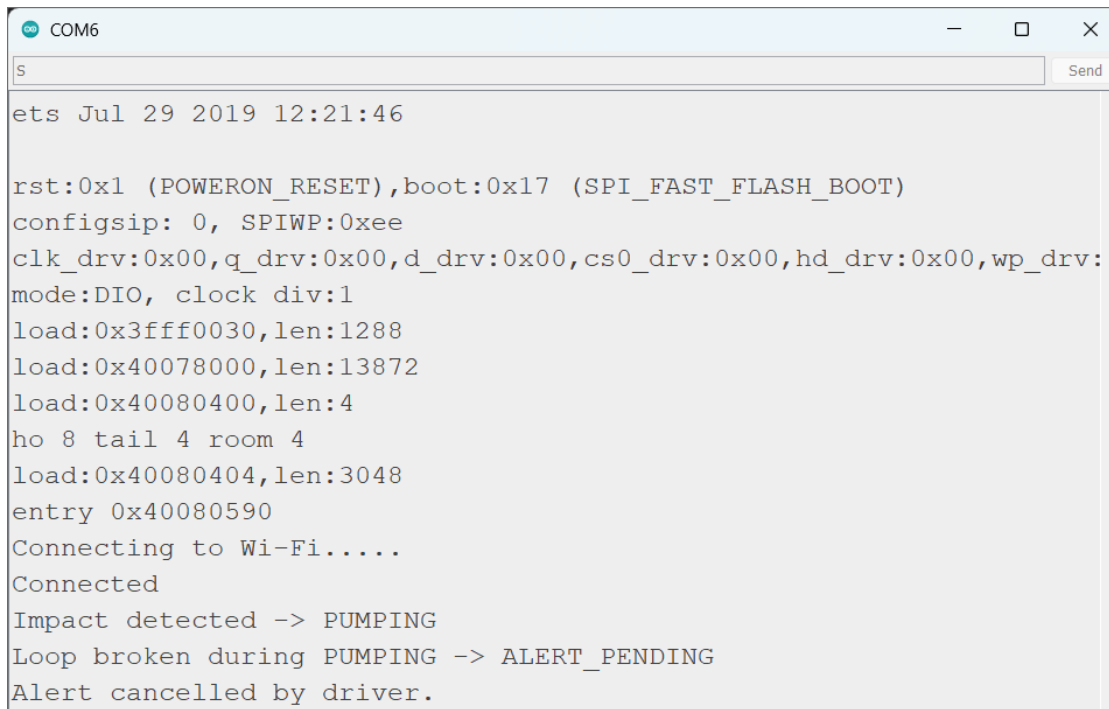
A reusable `debounce()` routine tracks a stable state and requires a 50 ms stable interval before accepting a change. The state machine uses the returned change indication to react to new events rather than simply reacting to a continuously asserted level. This is particularly important for the YL-99 impact input, where the firmware explicitly starts the pump on a new rising edge to HIGH.

13.3 GPS Caching

The firmware maintains `lastLatitude` and `lastLongitude`. When `gps.location` is valid, fresh coordinates are used; otherwise the previously cached coordinates are retained. This prevents the alert routine from immediately reverting to zero coordinates when a valid position is temporarily unavailable.

13.4 Error/Status Reporting

The firmware prints operational milestones to the serial monitor, including Wi-Fi connection, impact detection, pump stop, loop-break transition, alert cancellation, GSM command responses, and Firestore/RTDB success or failure. The supplied serial captures provide visible confirmation of both the alert and cancellation paths.



```
ets Jul 29 2019 12:21:46

rst:0x1 (POWERON_RESET),boot:0x17 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:
mode:DIO, clock div:1
load:0x3fff0030,len:1288
load:0x40078000,len:13872
load:0x40080400,len:4
ho 8 tail 4 room 4
load:0x40080404,len:3048
entry 0x40080590
Connecting to Wi-Fi.....
Connected
Impact detected -> PUMPING
Loop broken during PUMPING -> ALERT_PENDING
Alert cancelled by driver.
```

Figure 13. Serial-monitor evidence of the driver-cancellation path.

14. Testing and Observed Results

14.1 Alert Path

The supplied serial capture shows: "Connected", "Impact detected -> PUMPING", "Loop broken during PUMPING -> ALERT_PENDING", followed by "Sending call/SMS + Firebase log..." and successful-looking AT/Firebase responses. This is direct evidence that the demonstrated firmware executed the intended trigger-to-alert sequence.

14.2 Cancellation Path

The supplied cancellation capture shows the same initial impact and loop-break sequence followed by "Alert cancelled by driver." This is direct evidence that the 10-second cancellation state can be exited through the button input without entering the transmission/logging branch.

14.3 Application Output

The ambulance application screenshots demonstrate both an empty incident list and a populated list with accident coordinates and response status. This supports the documented cloud/application side of the prototype.

15. Technical Analysis and Design Considerations

Event confirmation strategy: The use of both an impact trigger and a physical circuit-break loop provides a two-stage event sequence. The YL-99 initiates the physical action, while the loop break is the condition that advances the firmware to the alert-pending state during pumping.

Timing strategy: The firmware uses non-blocking elapsed-time checks for the pump duration and cancellation window at the state-machine level. The GSM helper itself uses explicit delays while executing modem AT commands.

Communication redundancy: Emergency communication is represented through two independent external paths: GSM call/SMS and Wi-Fi/Firebase application logging. The project therefore does not rely exclusively on the mobile application channel.

Traceability: The combination of firmware messages, hardware diagrams, PCB artwork, prototype photographs and application screenshots gives a traceable link between the logical design and the demonstrated implementation.

Hardware modularity: GPS, GSM, and actuator interfaces are separately connected to the ESP32 through dedicated pins/UARTs, which supports modular testing of the major functional blocks.

16. Limitations and Engineering Constraints

- The supplied firmware contains placeholder Wi-Fi credentials, Firebase identifiers and phone numbers; these are not production secrets and must be replaced with valid deployment configuration before use.
- The supplied motor control comment explicitly notes that a proper transistor/driver should be used; the schematic/circuit documentation shows the L298N driver used in the prototype.
- The firmware assumes Wi-Fi connectivity during startup: `setup()` waits in a loop until Wi-Fi reports `WL_CONNECTED`.
- The GSM command sequence and SIM800L response handling are implemented with fixed delays rather than a fully asynchronous modem state machine.
- The GPS data used for the alert is the current valid fix or the last cached coordinates; the supplied firmware does not implement a separate "fix quality" or maximum-age policy.
- The project evidence demonstrates a prototype and PCB design, but the supplied materials do not establish automotive qualification, environmental validation, crashworthiness, functional safety certification or field deployment readiness.

17. Future Improvements

- Add a robust modem command/response state machine with explicit timeout and retry handling.
- Add GPS fix-quality checks, age limits and fallback behavior before issuing a location-based alert.
- Add persistent local event storage so an accident can be buffered when Wi-Fi is unavailable.
- Implement authenticated and least-privilege cloud credentials with deployment-specific configuration outside the source code.
- Improve PCB power integrity, protection, connectorization and mechanical packaging for field testing.
- Add repeatable test fixtures for impact trigger, loop-break event, cancellation, GPS acquisition and GSM delivery.
- Add formal validation of false-trigger resistance and timing under representative vibration and electrical-noise conditions.

18. Conclusion

Based on the supplied firmware, README, electrical documentation, PCB artwork, prototype photographs, serial-monitor evidence and application screenshots, AutoCrash SOS is an ESP32-centered embedded accident-alert prototype that integrates collision sensing, physical actuation, event confirmation, driver cancellation, GPS-based location reporting, GSM emergency communication and Firebase-backed ambulance notification. The firmware implements a clear three-state control flow and the provided evidence demonstrates both the alert and cancellation paths. The hardware and application artifacts show the corresponding physical and software integration layers.

Appendix A. Firmware Pin Map

GPIO	Signal	Configuration
GPIO 16	GPS_RX	UART2 receive from GPS
GPIO 17	GPS_TX	UART2 transmit to GPS
GPIO 4	SIM800L_RX	UART1 receive from GSM module
GPIO 5	SIM800L_TX	UART1 transmit to GSM module
GPIO 22	BUZZER_PIN	Buzzer output
GPIO 23	BUTTON_PIN	Push button, INPUT_PULLUP; pressed = LOW

GPIO	Signal	Configuration
GPIO 14	MOTOR_PIN	Air-pump driver output
GPIO 27	SWITCH_PIN	YL-99 impact input; active HIGH
GPIO 33	LOOP_PIN	Circuit-break loop; intact LOW, broken HIGH

Table A1. Firmware pin map.

Appendix B. Key Firmware Parameters

Parameter	Value	Meaning
PUMP_DURATION_MS	6000 ms	Maximum air-pump runtime per detected impact
CANCEL_WINDOW_MS	10000 ms	Driver cancellation interval
DEBOUNCE_MS	50 ms	Stable input change interval
GPS baud	9600	GPS UART baud
SIM800L baud	9600	GSM UART baud
Serial monitor baud	115200	USB serial diagnostic baud
NTP offset	19800 s	IST offset used by configTime()

Table B1. Key firmware parameters.

Appendix C. Supplied Project Assets

- autocrash.ino
- code.docx
- README.md
- architecture.png
- system_flowchart.png
- circuit_diagram.png
- Schematic_Autocrash.png
- PCB_Autocrash.png
- Autocrash_3D_PCB.png
- prototype_setup.jpg
- pcb_backside.jpg
- serial_monitor_alert.png

- serial_monitor_cancel.png
- ambulance_app_1.jpg
- ambulance_app_2.jpg